Enrollment No: 250843116006

pr_1 - Study the Data Analytics lifecycle and perform data acquisition from CSV, Excel, and JSON files using Pandas

Step - 1 : Import Pandas

```
# Load the pandas library (pd is the usual short name)
import pandas as pd
```

```
# Student data stored in a dictionary: column name -> list of values
data = {
    "ID" : [101, 102, 103, 104, 105, 106, 107],
    "Name" : ["Rahul", "Priya", "Amit", "Riya", "Karan", "Krisha", "Harry"],
    "Age" : [20,21,19,20,22,24,28],
    "City" : ["Surat", "Navsari", "Valsad", "Bharuch", "Surat", "Gandhinagar", "Ahmedabad"],
    "Marks" : [85, 92, 78, 88, 81, 84, 90]
}
```

Step - 2 : Create Datafrme

```
# Turn the dictionary into a table (DataFrame) and show it
df = pd.DataFrame(data)
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with `df` New interactive sheet

Step 3 : Save Data as CSV File

```
# Write the table to student.csv
df.to_csv("student.csv", index=False)
print("CSV file created successfully")
```

```
CSV file created successfully
```

Step 4 : Save Data as Excel File

```
# Write the table to student.xlsx, again without the row numbers
df.to_excel("student.xlsx", index=False)
print("Excel file created successfully")
```

```
Excel file created successfully
```

Step 5 : Save Data as JSON File

```
# Write the table to student.json as a list of records, indented by 4 spaces
df.to_json("student.json", orient="records", indent=4)
```

```
print("JSON file created successfully")
```

```
JSON file created successfully
```

Step 6 : Read the CSV File

```
# Load student.csv back into a new DataFrame and show it
df_csv = pd.read_csv("student.csv")
df_csv
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with `df_csv` New interactive sheet

Step 7 : Read the Excel File

```
# Load student.xlsx back into a new DataFrame and show it
df_excel = pd.read_excel("student.xlsx")
df_excel
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with `df_excel` New interactive sheet

Step 8 : Read the JSON File

```
# Load student.json back into a new DataFrame and show it
df_json = pd.read_json("student.json")
df_json
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with `df_json` New interactive sheet

Step 9 : Display First Five Records

```
# head() shows the first 5 rows
df_csv.head()
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81

Next steps: Generate code with df_csv | New interactive sheet

Step 10 : Display the Last Five Records

```
# tail() shows the last 5 rows
df_csv.tail()
```

	ID	Name	Age	City	Marks
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Step 11 : Display Dataset Info

```
# info() gives row count, column types, non-null counts and memory used
df_csv.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      7 non-null      int64
 1   Name    7 non-null      object
 2   Age     7 non-null      int64
 3   City    7 non-null      object
 4   Marks   7 non-null      int64
dtypes: int64(3), object(2)
memory usage: 412.0+ bytes
```

Step 12 : Display Statistical Summary

```
# describe() gives count, mean, std, min, quartiles and max of number columns
df_csv.describe()
```

	ID	Age	Marks
count	7.000000	7.000000	7.000000
mean	104.000000	22.000000	85.428571
std	2.160247	3.109126	4.961759
min	101.000000	19.000000	78.000000
25%	102.500000	20.000000	82.500000
50%	104.000000	21.000000	85.000000
75%	105.500000	23.000000	89.000000
max	107.000000	28.000000	92.000000

Step 13 : Display Dataset Shape

```
# shape = (number of rows, number of columns)
df_csv.shape
```

```
(7, 5)
```

Step 14 : Display Column Names

```
# List all column names of the table
df_csv.columns
```

```
Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

Step 15 : Display Data Types

```
# Check the data type of every column (int64 = number, object = text)
df_csv.dtypes
```

	0
ID	int64
Name	object
Age	int64
City	object
Marks	int64

dtype: object

Step 16 : Check for Missing Values

```
# Count empty (NaN) values in each column
df_csv.isnull().sum()
```

	0
ID	0
Name	0
Age	0
City	0
Marks	0

dtype: int64

Step 17 : Display a single column

```
# Pick one column (Name) - it comes out as a Series
df_csv["Name"]
```

	Name
0	Rahul
1	Priya
2	Amit
3	Riya
4	Karan
5	Krisha
6	Harry

dtype: object

Step 18 : Display Multiple Columns

```
# Pick two columns (Name and Age) - result is a smaller DataFrame
df_csv[["Name", "Age"]]
```

	Name	Age
0	Rahul	20
1	Priya	21
2	Amit	19
3	Riya	20
4	Karan	22
5	Krisha	24
6	Harry	28

Step 19 : Filter Records

```
# Keep only the rows where Marks is more than 85
df_csv[df_csv['Marks'] > 85]
```

	ID	Name	Age	City	Marks
1	102	Priya	21	Navsari	92
3	104	Riya	20	Bharuch	88
6	107	Harry	28	Ahmedabad	90

Step 20 : Calculate Average Marks

```
# Average (mean) of the Marks column
df_csv['Marks'].mean()
```

```
np.float64(85.42857142857143)
```

Step 21 : Sort Data

```
# Arrange rows by Marks, highest first
df_csv.sort_values(by='Marks', ascending=False)
```

	ID	Name	Age	City	Marks
1	102	Priya	21	Navsari	92
6	107	Harry	28	Ahmedabad	90
3	104	Riya	20	Bharuch	88
0	101	Rahul	20	Surat	85
5	106	Krisha	24	Gandhinagar	84
4	105	Karan	22	Surat	81
2	103	Amit	19	Valsad	78

Step 22 : Display Df

```
# Print the table as plain text
print(df_csv)
```

```
    ID    Name  Age         City  Marks
0  101   Rahul   20        Surat     85
1  102   Priya   21      Navsari     92
2  103    Amit   19       Valsad     78
3  104    Riya   20      Bharuch     88
4  105   Karan   22        Surat     81
5  106  Krisha   24  Gandhinagar     84
6  107   Harry   28    Ahmedabad     90
```